



Certified Kubernetes Administrator: Using kubeadm to Install a Basic Cluster

Course Overview

Course Overview

Hi, everyone! My name is Anthony E. Nocentino, Enterprise Architect and Founder of CENTINO SYSTEMS. Welcome to my course, Certified Kubernetes Administrator: Using kubeadm to Install a Basic Cluster. Are you a Systems Administrator that needs to build and administer Kubernetes clusters? If so, then this is the course for you. In this course, I will introduce you on how to build clusters using kubeadm. We'll look at system requirements, dive deep into the installation process, and then build a basic cluster together using kubeadm. By the end of this course, you'll know the system requirements for Kubernetes, cluster design concepts, you'll have a deep knowledge of the installation process, and you'll also know how to build a basic cluster using kubeadm. Before beginning this course, you should be familiar with Kubernetes and containers' architectural concepts and be familiar with the Linux operating system and administering at the command line. You should also have a firm understanding of TCP/IP based network. I hope you'll join me to continue your journey to learn Kubernetes in the course Certified Kubernetes Administrator: Using kubeadm to Install a Basic Cluster here at Pluralsight.

Using kubeadm to Install a Basic Cluster

Module Overview

Hello, this is Anthony E. Nocentino, and welcome to my course, Using kubeadm to Install a Basic Cluster, part of the Certified Kubernetes Administrator learning path. In this course, you'll learn how to deploy a Kubernetes cluster using kubeadm and some basic operation fundamentals. Let's get started. Let's take a look at what we're going to cover in this course. First up, in this module, we're going to start off with some installation considerations. Basically, what you need to know before you install Kubernetes. Then we're going to look at the installation overview. And after that, we're going to dive deep into the process of how we actually bootstrap a Kubernetes cluster. And once we have all of that, we'll sit down and we will learn how to install our first Kubernetes cluster together with kubeadm.



Installation Considerations

Let's start off this module with discussing some installation considerations. What this boils down to is where you're going to install Kubernetes. And first up is the cloud. And here there are really two major use cases when deploying Kubernetes in the cloud. You can deploy Infrastructure as a Service virtual machines, and then deploy a Kubernetes cluster on top of that. With IaaS-based deployments, the networking, the hypervisor of the infrastructure isn't something you have to worry about, but you do have to worry about the operating system, patching it, and then installing and maintaining Kubernetes itself as software on those virtual machines. And then there's PaaS offerings, or Platform as a Service offerings. Kubernetes is available to you as a managed service and all of the major cloud providers. You could simply go ahead and consume Kubernetes as a service, and you don't have to worry about maintaining that underlying infrastructure, patching operating systems, or even maintaining the installation of Kubernetes in your cluster. That's all handled for you by your cloud provider. Another deployment option for Kubernetes is on-premises. You can install Kubernetes on bare metal physical machines or using virtual machines. Now that leaves us with one giant question. Which one do you choose? Well, that's really dependent on a couple of things. What I like to think is that it's dependent upon the skill sets of the organization and the strategy of the organization. If you and your teams are cloud-ready and you're already in the cloud and you're moving forward as fast as you can, well, yeah, that makes a lot of sense to deploy in one of the cloud offerings, perhaps as an IaaS or as a PaaS offering. But if you want to stay on-prem, that's fine, too. Again, it goes to the skills and the strategy of the organization. Take these things into consideration when you're working out your installation considerations for Kubernetes. Now, this course is going to be about the installation and configuration of Kubernetes, and so we're going to focus a lot on installing Kubernetes manually ourselves in virtual machines.

Installation Methods

Now, let's look at some installation methods for how we can get Kubernetes up and running. First up is desktop installations. A desktop installation of Kubernetes is great for development environments. It's just playing around to learn how to work with Kubernetes. Primarily, I use Docker Desktop for Mac, but I just want to experiment with things when I'm teaching workshops or when I just want to kick the tires inside of the Kubernetes platform rather than using a full-blown installation or some managed service in the cloud. There is also a desktop version of Kubernetes for Docker for Windows as well. The installation method that's become the standard for installing and bootstrapping a Kubernetes cluster is a tool called kubeadm. This is a package that you can install, and with



kubeadm, you can bootstrap a cluster and get it up and running in a very fast way. This is the installation method that we will use in this course. And then finally, as we introduced a second ago, there are cloud scenarios where we can deploy either an IaaS or PaaS offerings in the various cloud providers. Do we want to just consume it as a service, or do we want to get things installed on top of virtual machines? So, lots of different ways to get access to Kubernetes. So go ahead and experiment with one of these, but in this course, we're going to focus on installing on virtual machines in our local on-prem system using kubeadm.

Installation Requirements

When we're installing Kubernetes on bare metal, or like we're going to do in this course, on virtual machines, we have some system requirements that we want to sort out before we get things up and running. And the first thing is you need Linux. And we're going to focus on Ubuntu virtual machines in this course, but certainly compatible with RHEL or any of the other major Linux distributions. Windows worker nodes are available, but that topic is going to be out of scope for this course. You'll need two CPUs in your system, as well as 2 GB of RAM. This is the bare minimum to bring a cluster up and running in your lab. If you're sizing a cluster for production workload, you'll need the appropriate resources to support your application workload in your cluster. Next, you need to disable swap on your system, and I'll show you how to do this in an upcoming demonstration. Swap has been supported since Kubernetes version 1.22, but in this series of courses, we're going to go with swap disable. In addition to the base system requirements that we just looked at, you'll need a container runtime, and you will need a container runtime that's compatible with the container runtime interface or CRI. Currently supported is containerd and CRI-O. For this course, we'll walk through the installation of containerd and build a cluster with that as our container runtime for our demonstrations. From a networking standpoint, we'll need network connectivity between all of the nodes in our cluster. Additionally, we'll also need to ensure that each system has a unique host name and also a unique MAC address.

Getting Kubernetes

So we've gone over the high-level installation scenarios, and we've gone through the system requirements, but what we haven't talked about is where do we actually get Kubernetes from? Well, Kubernetes is maintained on GitHub. And so if you go to github.com/kubernetes/kubernetes, here's where you'll find the living and breathing Kubernetes project. So honestly, if you're up for it, you can go ahead and contribute to this project yourself. It's a very, very active and vibrant community. In addition to just the raw software that's available on the website, there is a ton, I mean, a ton of deep dive documentation available for you



to learn how things work at the deepest level. In fact, if you're up for it, you can go read the code yourself. Now for this course, and for most of the production systems that I support, I use the Linux distribution repositories, both yum or apt, depending upon which Linux distribution I'm using. In this series of courses, we'll be using Ubuntu. And so we're going to get Kubernetes from an APT repository

Building Your Own Cluster

Now that we have the software in our hands, it's time to learn the steps that we need to take to build our first Kubernetes cluster together. And, well, step one is to install and configure a container runtime and the Kubernetes packages. Today in this course, we're going to learn how to install Kubernetes packages and use containerd as our container runtime. Then once we get the packages installed, we need to create a cluster. What this means is using kubeadm. We're going to bootstrap that first control plane node and get those critical cluster components up and running. Things like the API server, the controller manager, etcd, and so on. With the control plane node up and running, we then need to configure our pod networking environment. In this series of courses, we're going to use an overlay network for pod networking in our cluster. Once we have our pod network up and running, we can then join additional nodes to our cluster as worker nodes that we can use to run our application workloads on. Let's look at some of the required software packages that we need to install to get started working with our first Kubernetes cluster. Again, we're going to be using Ubuntu in this series of courses, so that means they'll be using the app package manager for package installation. First up, we need to install a container runtime, and we're going to use containerd in this series of courses. But as we discussed a moment ago, in our upcoming demo, we'll go through the process of installing and configuring containerd together. Next up is the kubelet. The kubelet is a thing that's going to drive the work on the individual nodes in our cluster, and that comes from a package named kubelet. Now, there's also a package kubeadm or kube-admin. It's a tool that's responsible for bootstrapping our cluster and getting all the cluster components up, running, and configured. And it's also the tool that'll be used to join additional nodes to our cluster. And then finally, there's kubectl, or kube control, or kube-"cutl," whatever you want to call it, but it's the primary command-line tool that we're going to use to administer our workloads in our cluster. Now, I do want to call out that you will install all four of these packages on all of the nodes in your cluster, regardless of if they're going to be a control plane node or a worker node.

Installing Kubernetes on VMs

Now, let's go ahead and look at the sequence of commands that we need to use to get and install Kubernetes on some Ubuntu virtual machines. And I do want to



point out that we want to do this on all of the nodes that we're going to have in our cluster, both control plane and worker nodes. First up, we need to install our container runtime. On an Ubuntu system, we'll do that with `apt-get install containerd`. Then when we get into the demos, we'll have some additional configuration steps that are needed to configure containerd to use the systemd C group driver. The next thing that we're going to do is we're going to add the GPG key for the apt repository where the Kubernetes packages live. Then we're going to execute this sequence of commands here to add the Kubernetes app repository to our local repositories lists. Now don't worry about writing this all down or taking screenshots. This will be available to you in the course downloads. We're going to do this in an upcoming demonstration together. I do want to point out that where I have that version variable, that would need to be updated for the version of Kubernetes that you want to run, and at the time of this recording, it's 1.29. A working example of this will be shown in the upcoming demos, and I'll keep the course downloads updated with the latest tested version for this course. Now, with the repository installed, we need to tell apt to update its package list, and we can do that with `apt-get update`. And then we'll use `apt-get install` to install the three remaining required packages, the `kubelet`, `kubeadm`, and `kubectrl`. Now, here's where we're going to deviate from the normal Linux package installation process. What we're going to do here is we're going to mark these packages with `apt-mark hold` for the four packages that we just installed. The reason for this is that we no longer want apt to maintain the upgrading of these packages. We're going to want to service these packages outside our normal security updates for the system so that we can control when we move between versions of Kubernetes independent of security patches. This is needed because Kubernetes has a defined upgrade process when moving between versions. We're going to cover that process in a course later in the series.

Lab Environment Overview

We just looked at the code level detail to install Kubernetes on some Ubuntu virtual machines. So let's zoom out a little bit now and look at the lab environment that we're going to use in this course and the remainder of the courses in the series. And so here you can see we have one control plane node and three worker nodes, and so that's going to be the topology of the cluster that we're going to construct together in the upcoming demonstrations and use throughout this series of courses. What we're going to have here is we're going to have `kubectrl` installed onto the control plane node, and so we're going to drive all of our work from that particular node when we work with our cluster. The version of Ubuntu that I'm using in the lab for these virtual machines is Ubuntu 22.04. Now, as versions of Kubernetes change and versions of Ubuntu change, I will continue to update the code into the course downloads to ensure that they work with the latest versions of Kubernetes and Ubuntu. If you experience any issues, please



reach out to me via email. Now, for our lab virtual machines, my virtual machines are VMware Fusion VMs. You can use pretty much any hypervisor that you want as long as you have full network connectivity between all of the nodes in your cluster. The minimum CPU requirements are 2 vCPUs, and you'll need at least 2 GB of RAM in each of these virtual machines. Now, I was a little generous with the virtual machines from a disk space standpoint because, honestly, that's a little bit harder to change than any of the other attributes in a VM. And so I've allocated 100 GB on each of the nodes in the cluster. Now, for these virtual machines, I'm going to go with a disabled swap, and I'm going to show you how to configure that in some upcoming demos. Now, I have all of the host names set for each of these individual nodes in a host file on each of the virtual machines with the corresponding names and IP addresses so I don't have to rely on DNS for name resolution between these virtual machines. And now as for host names, here we go, c1-cp1 for cluster 1, control plane 1. And the IP address that I'm using in my lab is going to be 172.16.94.10. You can use whatever IP scheme you want to use in your lab. Next up is c1-nod1, so pretty easy naming convention that we're going with here. And its IP address is going to end in .11, then c1-node2, ending in .12, c1-node3 ending in .13. And so that's our cluster, a control plane node, and three worker nodes.

Demo: Installing and Configuring containerd

All right, it's taken us a long time to get here, but it's time to do our first demo together. In this demo, what we're going to do is we're going to install containerd, kubelet, kubeadm, and kubectl. And then once we have those packages installed, we're also going to look at some systemd units for these packages to understand how they operate with the underlying systemd system. For our demos, I'm going to use the VS code, and then what I'm going to do is at the top of the screen, I'm going to highlight the code that I want to run, and we'll see the output at the bottom of the screen so we can see both the command and its output at the same time. I've also deployed the four virtual machines that we just looked at in the presentation portion of the course and have an SSH connection into c1-cp1. Let's get started. All right, so here we are logged into c1-cp1. And the first thing that we're going to do here is go ahead and check on the configuration of our swap file. And we can verify if our swap is turned off or on by using the command here on line 14, `sudo swapoff -a`, so when I run that code there, if I get no output, then we know that our swap is disabled. How do I disable the swap? Well, that's what I did inside of our `fstab`. So I'm going to go ahead and open up `/etc/fstab`, and you can see in the output at the bottom here, I've commented out the line for our swap config by putting that hash mark in the front. So I go ahead and save that out and I reboot the system, that's going to disable our swap file. Now, what they want to call out is it's possible that this configuration is going to change. This is the second version of this course that I've recorded for Pluralsight, and this particular installation



sequence has changed many times. I will always keep the course analysis up-to-date with the latest method. And so knowing that, let's look at how we can install both containerd and our Kubernetes packages in this demo. So the first thing that we need to do before we install containerd is to configure some operating system prerequisites. We need to load some modules and some sysctl parameters. And so that's what we're doing here on line 26. There's a here doc that's going to add the overlay and br_netfilter modules to the startup of this system. And we're going to need those at runtime right now, and so that's what I'm doing here on lines 31 and 32. I'm loading those modules in our system. Once those are loaded, I also need some sysctl parameters defined. So let's go ahead and run this code here on line 35 to configure those, and then on line 43 to load them into memory right now. That first line there being a here doc that's going to put them into k8s.com, which will persist across reboots, and on line 43 here, the sysctl parameters, loading them right now in demand, so they're available to us for this installation. With those prerequisites set, it's time to install containerd, and that's what we're doing here on line 47, `sudo apt-get install -y containerd`, we'll run that code there, and I'll begin the containerd installation process. And then once that's completed, let's go ahead and configure containerd. The first thing that we need to do is make a directory for a configuration file in `/etc/containerd`, and that's what we're doing here on line 51. Once we have that created, we'll use the command `containerd config default` to generate a default configuration file. So here on line 52 we can see just that, `sudo containerd config default`. It's going to generate a configuration file that we're going to pipe into `sudo tee` and write a configuration file out in `/etc/containerd/config.toml`. Run that code there, and we'll have that configuration file created. With the default configuration file created, what we need to do now is make one adjustment to that file so that we load the SystemdCGroup driver. And so on lines 60 through 64, we can see the insides of those files here in our code, and on line 64, we see `SystemdCGroup = true`. Well, the default is false, and so to change that real easily I have a set command on line 67 that will swap false to true for you. So once you run this code here on 67, we'll have the appropriate configuration in our containerd configuration file. We want to verify that change is made on line 71. We use `grep` to see if that value is changed in a file, and there in the output at the bottom, we see `SystemdCGroup = true`. So now that's properly configured, we do need to restart containerd, and we'll do that here on line 75 `sudo systemctl restart containerd`.

Demo: Installing and Configuring Kubernetes Packages

So now that containerd is installed and configured, it's time to configure our app repositories for the Kubernetes repositories and install the required packages for Kubernetes, so `kubeadm`, `kubelet`, and `kubectrl`. The first thing that we'll do is we'll install some required packages for this process. So on line 82, I have `apt-get install, apt-transport-https ca-certificates, curl, and gpg`. Once those are



installed, now it's time to get the k8s.io apt repository gpg key and install that locally on a machine so that we can trust the repository. On line 83 we have that code, and I want to call out in the middle of that code v1.29. If you recall in the presentation portion of the course, add a variable there, version. Well, here is an example of version 1.29. This will likely change over time as Kubernetes releases come about, next one being 1.30, and so on. So, update that to the latest and greatest version that you want to work with, and I'll keep the course, update it, and test it against those newer versions. So for today, we're going to use 1.29. So let's go ahead and run this code here to get that apt repositories gpg key and have that trusted locally. Once that's done, on line 87, we'll now configure that apt repository in our local repositories. And again, the same pattern. In the presentation portion of the course, I had a version variable here, and on the code here on line 87, we see v1.29. So let's go ahead and run this code here to add that repository to our local repository's lists. So with that repo trusted and now configured, let's go ahead and update our package metadata on this particular host with `sudo apt-get update`. And once that's completed, I want to take a second to look at the available versions of the Kubernetes packages that are available for us, and we'll look specifically at the kubelet. So when I run this code here on line 92, what that's going to do is output all the possible versions of kubelet that are available to me. So in the output at the bottom, we see 1.29.0, 1.29.1, and 1.29.2. Well, those are the particular versions of Kubernetes that are available to us to install. And so what we're going to do next is install a specific version on our system. I'm going to install version 1.29.1. And that's what we have here on line 98, I'm defining a variable, 1.29.1-1.1, so that exact package version that we see listed in the output at the bottom. And the reason why I'm doing this here today is in future courses, we're going to upgrade to a newer version of Kubernetes. And so this way in the sequence of demos that work through in this course, you'll have a version to upgrade to later. And so let's go ahead and run this code here on line 98 to populate that version variable. And on line 99, what we're going to do is we're going to install the exact versions of our softwares. And so walking from left to right, looking at the code, `sudo apt-get install -y kubelet=VERSION, kubeadm=VERSION`, and so on for kubectl. And that's going to install those exact versions of those packages for us. Once that's completed, we'll use `apt-mark hold` to hold those packages on these particular versions. And so we update the system software with `apt-get upgrade` to maintain the security patches and so on. For our system, these particular packages will always stay on the versions that they're at right now. We'll look at upgrades later on in an upcoming course. Now, if you want to install the latest and greatest Kubernetes packages, I have the code here on lines 104 for you to do just that. So now that we have everything installed, let's look at our systemd units. I want to look at two specifically, the kubelet and containerd because I want you to know the state that they're in right now before we bootstrap our cluster. Because all we've done so far is just install the packages. And so `sudo systemctl status kubelet.service`. Run that code here, and I want to call out in the output here, we can see that the unit is loaded, and then it's enabled, and the preset is set to enabled. So when



the system reboots, the kubelet will automatically restart. I do want to call out that in the active field, we see that it's inactive (dead) because we have not bootstrapped our cluster yet. There's no workload on this particular node for that kubelet to be active. And in an upcoming demo, we'll look at what that looks like once we've bootstrapped our cluster. And so now that we know that inactive (dead) is the correct state that we want to be in in this phase of our installation, let's look at the status of containerd. We can do that with `sudo systemctl status containerd`. Here, containerd is up and running and active, and we see that there in the output active running. It's also set to enable, so when this system reboots, containerd will automatically restart. So let's go ahead and break out of this output here and clear our screen. And so that's a wrap for this demo where we've installed containerd and the required packages for Kubernetes.

Bootstrapping a Cluster with kubeadm - Control Plane Node

With the software packages installed, it's now time to start the process of bootstrapping or creating a cluster with kubeadm. Kubeadm is a tool that we can use to manage the process of creating our cluster. It's going to walk through various phases and build and configure our cluster for us, and right now we're going to walk through each one of those phases together so that you know what's really going on during the creation of a Kubernetes cluster. Now, to Bootstrap a cluster at the command line, you'll type `kubeadm init`, and that's going to begin the process of creating your cluster. The first phase is pre-flight checks. What kubeadm is going to do for you here is execute a series of checks that will help ensure a successful cluster creation. So things like ensuring that you have the right permissions on the system that you're working with. It's also going to pull the container images of the various parts of the control plane, which are going to run as pods on the control plane node. It's also going to verify that you have the appropriate system resources on your server in terms of CPU and memory, and another important pre-flight check that it executes is, is there a container runtime on this system, and is it up and running? If any of these checks fail, kubeadm will report the error and stop the cluster creation process. The next thing that kubeadm does for you is creates a certificate authority for you. Kubernetes uses certificates for both authentication and encryption, and we're going to look very close at this in an upcoming slide. In the next phase, what kubeadm does is it generates kube config files for you for the various cluster components of Kubernetes so that they can locate each other on the network and authenticate against the API server. We're also going to look at these very closely in an upcoming slide. The next phase is generating static pod manifests. Static pod manifests are going to be generated for each of the control plane pods. What's going to happen here is they're going to be created and stored on the local file system in a location that the kubelet monitors. And if the



kubelet finds a pod manifest there, well, it's going to start up the pods that are defined in that manifest. We're going to look at this in the upcoming slide, and we're also going to look at this deeply in an upcoming demo. Now, after the static pod manifests are generated, kubeadm waits for the control plane pods to start up and get running, so the API server, etcd, and the controller manager, and so on will all be started up as pods on our control plane node. Now what kubeadm will do is taint the control plane node. And we discussed this a little bit when we looked at pod operations in a previous course in the series. What this means is when Kubernetes schedules pods, it will never schedule user pods onto the control plane node. It will only ever schedule system pods to run onto the control plane node. Now, kubeadm goes on to generate what's called a bootstrap token. A bootstrap token is used for joining additional nodes to the cluster. And now in the final phase, kubeadm will go on to start up the add-on pods, so things like DNS and also the kube-proxy pod. The process that we described here is the default process if you just use kubeadm init with no parameters. This bootstrapping process is very customizable, and you can build more complex or customized clusters using kubeadm command-line parameters, or even a YAML configuration manifest to describe your cluster's configuration.

Understanding the Certificate Authority's Role in Your Cluster

It's time now to zoom in on those facets of the kubeadm in a process that I called out a second ago, and we're going to start the conversation with the Certificate Authority. Kubeadm init by default will create a self-signed Certificate Authority for you. If desired or perhaps required in your organization, you can tell kubeadm init to integrate with an external PKI, or public key infrastructure. The Certificate Authority that's created by kubeadm is used to secure cluster communications. It's going to generate certificates that are used by the API server to encrypt the HTTP stream that is the API server's communication protocol, so that's going to be HTTP over TLS, or, in other words, HTTPS. In addition to securing cluster communications, the Certificate Authority is used to generate certificates for the authentication of users and cluster components, like the Scheduler, Controller Manager, and the kubelets to authenticate the request to the API server. So we'll have strong authentication methods for users that have to operate the cluster, any of the cluster components, and also for the kubelets on the nodes in our cluster, since each of these things use the certificates to authenticate and verify their identity. The Certificate Authority files are going to be stored in `/etc/kubernetes/pki` on the control plane node and will be distributed to each node in your cluster when you join additional nodes to your cluster using kubeadm init. This distribution process ensures that the nodes trust the self-signed Certificate Authority. For more information on how to control or customize your kubeadm in a process, check out this link here. This article includes many of the customized parameters for kubeadm init, including



specifying external PKI configurations.

kubeadm Created kubeconfig Files and Static Pod Manifests

Next up in our deep dive into the kubeadm init process is kubeadm Created kubeconfig files. What a kubeconfig file is is a configuration file that defines how to connect to your cluster API server. Inside a kubeconfig file, you're going to find the certificates used for client authentication, and you're also going to find the API server's network location, usually its IP address, or a DNS name. Often included in a kubeconfig file is the CA certificate of the CA used to sign the certificate of the API server. This way the client can trust the certificate presented by the API server. Kubeadm creates a collection of kubeconfig files, and those live in `/etc/kubernetes`. And let's look at each one of those kubeadm Created kubeconfig files now. The first one that's generated is `admin.conf`, which is a Kubernetes cluster administrator account, essentially a super user inside of a Kubernetes cluster. We're going to use this to authenticate to our cluster today. There's also a `super-admin.conf`, which is a special account that can be used to completely bypass the authorization layer in your cluster. This is a break glass in case of emergency thing if your authorization layer is broken. Kubeconfig files are also generated for the controller manager and the scheduler to tell each of these control plane components where the API server is and which certificates you use to authenticate to the API server with. On your control plane node, you'll also have a kubeconfig file for the kubelet, `kubelet.conf`. This is used to locate the API server and present the correct client certificate for authentication. These kubeconfig files are generated by kubeadm and are used by these components to locate and authenticate to the API server. Later in a series of courses, we'll look at how to create kubeconfig files for users to locate and authenticate to the cluster, and we'll also learn how to set appropriate permissions for those users. Now let's take a closer look at what kubeadm init does for us with static pod manifests. A manifest describes a configuration, and in this case, it's going to describe the configuration of a pod. And so what kubeadm is going to do, it's going to generate static pod manifests for us for the control plane components that it needs to bootstrap a cluster and get it up and running. These static pod manifests are going to live in `/etc/kubernetes/manifests`, and it's the job of kubeadm init to generate a static pod manifest for the core cluster components, and that includes etcd, the API server, the Controller Manager, and the Scheduler. And so kubeadm init generates the static pod manifests for each of the control plane pods, and they live in that special directory. It's the job of the kubelet to watch this directory on the file system, and for any static pod manifests it finds in this directory, it'll start up the pods described in those manifests. And in this process here, it's the control plane pods. This is how a cluster starts up after a reboot as well. The kubelet starts by the systemd daemon on the node on boot. And then when the



kubelet finds the manifests in this directory, it will start up the control plane pods defined in those static pod manifests. Now, you might be thinking, why do we need this special configuration to start up these pods? Well, this is what enables the start up of those core cluster components as pods, well, without a cluster up and running.

Pod Networking Fundamentals

That wraps up the conversation about kubeadm init and the closer look at its various phases. Now let's shift the focus over to the last thing that we need to talk about before we actually stand up our Kubernetes cluster, we're going to talk about pod networking. Let's say we're designing the cluster. We have the requirements of the Kubernetes networking model that we introduced earlier in the series of courses. We need to ensure that we have a single un NATed IP address per pod and that all pods and all nodes have reachability between each other. Now, we could, of course, configure direct routing. And in this case, we need to configure our networking infrastructure underneath our pods and nodes to support the full IP reachability between these pods and nodes without NAT using real IP addresses. And sometimes that's just not feasible. Perhaps you're in a cloud scenario, or you're in a scenario where you don't have control over the underlying infrastructure, and that's where overlay networking comes in, sometimes called software-defined networking. With overlay networking, what happens is we get the appearance of a single layer 3 network that all the pods can communicate on. And some of the techniques that are used to make this happen include tunneling and encapsulation of those IP packets. And it will be the responsibility of the overlay network to facilitate for the communication of those packets between the pods in an unchanged way. Now, some of the overlay networks that are available to us include Flannel, Calico, and Weave Net. These are overlay networks, each of which have different capabilities and features that may be interesting to you as you deploy your overlay network. In our demos today, we'll be using Calico for our pod network. When deploying a pod, it will provide the IP address management for the pods deployed. It's imperative that the network address ranges that you choose not overlap with any other networks in your network infrastructure. So work with your network engineering teams to find an appropriate address range to use for your pod network. For a full discussion of overlay networking, I encourage you to check out this link [here](#). You'll find many more overlay networks available and other networking solutions that you can use for your pods and nodes so that they can all communicate together. This discussion here is really just scratching the surface for what you need to know about networking to get a cluster up and running, and we will have several courses dedicated to K and it is networking coming up in this series of courses.



Creating a Control Plane Node

With all the preliminaries out of the way, it's time to check out the code to use `kubeadm` to bootstrap a control plane node in our Kubernetes cluster. The first thing that we need to do is to download a YAML manifest that describes our pod network that we want to deploy, and that's going to be `calico.yaml`. This is the actual deployment manifest for the pod network, and inside that file is where you'll specify the pod network IP address range. We'll look how to configure that in our upcoming demo. I also want to call out that if this URL changes, I'll update the course code downloads with the new URL. The next thing that we need to do is to execute `kubeadm init`, kicking off the `kubeadm init` process that we just walked through together. Once `kubeadm init` finishes, all the control plane pods will be up and running. `Kubeadm init` will also print out the commands and parameters needed to join additional nodes to your cluster, and it will also print out how to configure a `kube/config` file for an administrative user. We're going to dive deep into the join command in a moment later on in this module. But for now, let's look at how to work with the `kubeconfig` file for our administrative user. The following steps will enable your currently logged in user to perform administrative functions on the cluster. And the first thing that we'll do is we'll create a directory for our `kube/config` file in our regular user's home directory. And then we'll copy the `admin.conf` `kube/config` file from `/etc/kubernetes` into that directory in our home directory. And then we'll adjust permissions on that file so that we can access this account with our current user. This `kube/config` file allows our currently logged in user to perform administrative functions on the cluster, and inside that `kube/config` file, you'll find the required certificates to authenticate to the cluster, and the network location of our clusters API server. The final step that we need to take after creating our control plane node with `kubeadm init` is to deploy a pod network, and we can do that with `kubectl apply -f calico.yaml`. This will read the deployment manifest file that we just downloaded, send it into the API server, and create the pod network in our cluster. Once the pod network is finished, our DNS add-on pods will start up, and we'll have a fully functioning control plane node.

Demo: Creating a Cluster Control Plane Node

Now, let's get into a demo and look at how we can create our first Kubernetes cluster together. Once that cluster is up and running, we're going to deploy a pod network, and then we're going to look at those `systemd` units again to see what changed from the last time when we installed the `kubelet` and how it's going to react differently now that we have a cluster up and running because of the static pod manifests created by `kubeadm init`. Then we're going to look at those static pod manifests from our control plane pods. Let's get started. All right, so here we are logged in the `c1-cp1`, and let's get started with the process of creating our



control plane node. The first thing that we're going to want to do here is to download the manifest for our pod network, and that's the code I have here on line 8. That's going to use `wget` to retrieve that `calico.yaml` manifest from the URL specified. Now, next thing we will do is we're going to crack open that file with `cat` and look inside this file for a string, `CALICO_IPV4POOL_CIDR`. So let's go ahead and hop down into the output at the bottom. We'll search for that string. And here in this commented code, you can see the value `192.168.0.0/16`. That's the default IPv4 CIDR range for the pod network. We're going to go with this default in our configuration today, but make that modification as appropriate to your network infrastructure. So let's go ahead and break out of them here, and go on next to the code on line 18 where we have `sudo kubeadm init`, which is going to bootstrap our cluster. We're specifying a specific version of Kubernetes here, as we did in the previous demonstration when we installed the Kubernetes packages. We're installing 1.29.1. Specifying the version here will allow you to upgrade to another version when we get to that course into the series, and so the parameter, `-kubernetes-version v1.29.1` is the version that we're going to run inside of this cluster. So let's go ahead and run this code here together to kick off the bootstrap process. Now, we walk through this process in great detail in the presentation portion of course, and so what we'll see in the output at the bottom is the `kubeadm init` process walking through each of those phases. So there at the bottom, we can see it's doing the pre-flight checks. And at this moment in time, it's actually pulling the container images for each of the control plane components from the appropriate container repository. All right, now with that completed, let's scroll up and look at some of the output in the `kubeadm init` process, so we can see it as it walks through each of the different phases of the `kubeadm init` process. So there we can see at the very first line `init using Kubernetes version 1.29.1`, which is what we specified in the command-line parameter for the `kubeadm init` process. It goes through the pre-flight checks. Then we can see in the next phase here, it's creating the CA certificate. Going and generating the CA certificate and key and then generating the certificates for each of the control plane components, API server, etcd, and so on. Going down a little bit further, we can see the phase where it creates the kubeconfig files for each of the kubeconfig files that we talked about in the presentation portion of the course, and those are going to be saved in `/etc/kubernetes`. At this point in time next, it goes and starts to create the static pod manifests, writing those out to `/etc/kubernetes/manifests` and getting one of those for each of the control plane components, API server, Controller Manager, and so on. At that point in time, those are written into the static pod manifest folder. The kubelet is then watching for those files to be written, and then we'll begin to start up that process of booting the control plane pods, and here we can see `wait-control-plane` while it's waiting for those pods to pull the container images and then start those pods up. Go down a little bit further. We then see the bootstrap token get generated, which will be used for other nodes to join this cluster. And then finally, we see the add-on pods, CoreDNS, and kube-proxy get applied configuration, so those will be available for us in a little bit. In



the output here, this is what you want to look for when you bootstrap your cluster. Your Kubernetes control plane has initialized successfully. Then we see the output to copy admin.conf into our home directory for the kubeconfig so that we could authenticate the cluster. And we also at the bottom here see the command to join any additional worker nodes to the cluster. We're going to revisit that in an upcoming demo. But for now, let's go ahead and move forward in this demo. So, I do want to point out that if you do want to run a newer version of Kubernetes and not specify the one like we did here, you just omit the Kubernetes version parameter. And that's the code that we have here on line 22. Now, moving forward, let's go ahead and copy admin.conf into our local users kubeconfig file, and that's the code that I have here on line 30 through 32. So let's go ahead and run that to create the .kube directory in our home directory. We'll then copy admin.conf out into our .kube directory, and then apply the appropriate permissions to access that file. And so now we have the ability to run kubectl commands with administrative privileges against our cluster. And that's what we're going to do here on line 37 and deploy our pod network with kubectl apply -f and specify in calico.yaml, which is the pod manifest, for our pod network. At the bottom there, you can see deployment.apps/calico-kube-controllers created, and a bunch of other resources created. So now that's going to go ahead and create the pod network in our cluster, and we're going to move forward and check the status of that deployment, and we can do that with kubectl get pods, specifying -all-namespaces, which will show us all the pods running in our cluster, even system pods. At the bottom here, we have a whole collection of pods that I want to walk you through. So this is the first time we're seeing the output of a kubectl get pods. And we've created a bunch of system pods, or control plane pods, and also our pod networks pods. And so let's look at the output at the bottom here. So from left to right, we define the namespace, which is kind of like a folder to hold resources in our cluster. They made the resource name, we have a column READY, which defines if the containers in the pod are up and running, and then the actual status of the pod. And so we see a couple of them are container creating and a couple of them are running. So everything's kind of getting started up at this point in time. We see if there were any container restarts, and also the age of the resource. Now we'll walk through this output to look at some of the pods. At the bottom there, we can see our control plane pods. We see kube-apiserver-c1-cp1. That's our API server's pod running on our control plane node, c1-cp1. We have a pod for controller-manager, we have a kube-proxy pod, and we also have a scheduler pod. Looking at the other pods in a list, there we see /etc, the CoreDNS, pods and the Calico networking pods on our control plane node. Let's go ahead, and if you want to, when you're running this command, if we do kubectl get pods -all-namespaces, if you want to see the pods kind of with a watch or like a tail, if you're familiar with that concept, we see that the pods are all up and running, but as they change state, this would update this output here for us in our cluster's output for our kubectl get pods command. So, repainting the screen as the pod's state changes. So it's going to break out of that with Ctrl+C. We see



that all of our pods are up and running. We can confirm that again with the `kubectl get pods`. So once you see this state, we can move forward. All of the pods are up and running, both the control plane pods and our pod networking pods. At this point in time, if we do a `kubectl get nodes`, we want to check the status of our control plane node. And it's a very important part here before we move forward is that the status is ready. We can see that in the output at the bottom. Reviewing all of the output, we see the node name, `c1-cp1`, which is our control plane node. We also see that its role is control-plane, and it's been up and running for about 6.5 minutes, and its version is 1.29.1. So now we have an up-and-running functioning control plane node with a pod network deployed.

Demo: Examining systemd Units Again and Static Pod Manifests

I want to take a few minutes now to review systemd units again. In our previous demonstration, we saw that the status of our kubelet was inactive and dead. If I go and I ask for the status of our kubelet now with `sudo systemctl status kubelet.service`, if I look at the output, now we see active and running. And the reason for that is our kubeadm init process created those static pod manifests and wrote them into the directory that the kubelet was watching. Once the kubelet so that those static pod manifests were there, it starts up, starts up those control plane pods, and there we have our control plane pods up and running. And if we reboot our system, that same process happens again. The systemd daemon starts up the kubelet service. it looks in the watch directory, and reads out those static pod manifests, starting up the control plane pods on reboot. And let's look at those static pod manifests in `/etc/kubernetes/manifests`. And so if I do a directory listing on that directory, what we'll see inside of there is a static pod manifest for each of the control plane pods' components. From left to right there, we see `etcd`, kube API server, kube Controller Manager, and kube Scheduler. And so this is the directory that the kubelet watches for the static pod manifests. If we look inside one of those static pod manifests with the `more` command, and we'll look `etcd.yaml`, inside of here, we can see the configuration of this particular pod. And so we see things like the API version, the kind is a pod. If we go down a little bit further, we'll see the pod spec, which is the way to define the configuration of a pod. And so these are all the configuration elements of `etcd`. We'll cover all of these details later on in another course where we talk about the maintenance and backup and restore of `etcd`. But for now know that this is where this configuration lives. And similarly on the API server, if we look at `kube-apiserver.yaml`, we also see the configuration of the API server in here. So let's look at some of this code for our API server. And so that's going to define things like the container image and also its configuration. Again, we'll dive into much more detail into that in an upcoming course. The final thing I want to call out to you in this demo here today is where the kubeconfig files live for the control plane pods, and that's going to be in `/etc/kubernetes`. So there's our



admin.conf, which is our cluster administrator. We have a Controller Manager, we have our local nodes kubelet config, we have our scheduler.conf, and also our super-admin.conf. And so this is the directory where all these kubeconfig files live for the various components of our control plane node. And so that's a wrap for this demo. We've gone ahead and successfully bootstrapped a control plane node, deployed a pod network, and we checked out where a couple things live inside of our node in terms of static pod manifests and kubeconfig files.

Adding a Node to Your Cluster

Nodes or worker nodes are where your user workloads will run in your cluster. To join a node to a cluster, you will first install the required Kubernetes software packages, so your container runtime, which in our case is containerd, kubeadm, kubelet, and kubectl. We then use kubeadm with the join parameter to start the process of joining a node to a cluster. Kubeadm join takes some parameters, such as the network location of the API server, a bootstrap token used to join a node to a cluster, and also the CA cert has to trust the certificate presented by the API server when kubeadm connects to that API server. We're going to look at some example code of this in a second. We then execute kubeadm join on the node that we want to join to the cluster. What's going to happen is the node is going to download the cluster information and cluster configuration metadata, then it's going to submit what's called a certificate sign-in request, or a CSR, into the API server to generate a certificate, which will be used by the kubelet on the node that we're joining to the cluster. This is used by the kubelet to authenticate to the API server. The CA is going to automatically sign that CSR in that certificate request, and then kubeadm join is going to download that certificate and store that on the file system of the soon-to-be-joined node. That certificate is going to live in /var/lib/kubelet/pki. Kubeadm join is then going to generate a kubelet.conf kubeconfig file for the kubelet in etc/kubernetes on the node. Inside that kubeconfig file is a reference to that client certificate that was just created and downloaded and the network location of the API server that we want to authenticate against for the cluster that we want to join this node to. The code for kubeadm join looks like this, kubeadm join, and then the DNS name or the IP address support of the API server, which in our lab is 172.16.94.10 on port 6443. The next parameter is the bootstrap token. This is effectively a time ticket or password for joining a node to a cluster. The next parameter is the ca-cert-hash. The ca-cert-hash is used here to establish a certificate chain of trust between the node joining the cluster and a certificate presented by the API server for HTTPS requests. In our last demo, you saw how kubeadm init prints this command and the required parameters to screen, and you can use that to join additional nodes to your cluster. But what do you do if that code goes away? Well, in our upcoming demo, I will show you how to use kubeadm to generate the entire join command on-demand so that you can use that information to join additional nodes to your cluster.



Demo: Adding a Node to Your Cluster

All right, so let's get into a demo and learn how to join a couple of nodes to our new cluster. All right, so here we are logged into c1-cp1. In this demo, what we're going to do is we're going to join a worker node to our existing cluster. And the first thing that we need to do is to log into our worker node with SSH. So that's what we're doing here on line 2, we're logging into c1-node1. Now recall our cluster is comprised of a control plane node, and it will have three worker nodes. We're going to install and configure one worker node here, c1-node1. You'll go ahead and install c1-node2 and 3 on your own later on in this exercise. And so let's go ahead and get started with configuring the base OS and installing the Kubernetes packages on this worker node. And so the first thing that we'll need to do is to confirm that our swap is disabled. And when I run `swap off -a` and get no output, that is the case. If I need to make a modification, I'd hop the DFS tab here and make the modification to disable the swap. Now, let's go ahead and move forward and configure the required modules for Kubernetes. That's what we're doing here on line 16 through 19 with the `here doc`. And we'll load those two modules at runtime right now with the code on lines 21 and 22, and jumping through this networking configuration here on 25, configuring these networking parameters on boot. With that `here doc` and then the code on line 32, they'll load those right now into the system. So now that we have the base OS configured, it's time to install containerd with `sudo apt-get install -y containerd`. And then once that's installed, we need to make the configuration change for the SystemdCGroup driver. That installation is finished. Let's go ahead and make our containerd configuration directory. We'll go ahead and generate that default configuration again with `containerd config default`. That's going to generate `config.toml` in our `/etc/containerd` directory. Just like we did before, we'll need to configure the SystemdCGroup driver in that config file, the code on line 56, swaps that out with the `set` command by setting the value from `false` to `true`. And before we move forward, let's confirm that that change was made with the code here on line 60, and it sure is. And at the bottom, we can see `SystemDcgroup = true`. Now that that's installed and configured, we'll restart containerd to make that configuration change take effect. So on this worker node, we also do need to install the Kubernetes packages, and that's what we have in this part of our demo here. We'll install some additional required packages, such as `apt-transport-https ca-certificates curl gpg`. We'll then install the k8s.io apt repository gpg key into our local key ring, and remember to update that version, the 1.29.2, whichever version it is that you're working with. With the gpg key installed, we'll go ahead and add that repository to our local repositories list, and we'll update our package metadata with `apt-get update` again. If you like, you could browse the available versions of the Kubernetes packages available with `apt-cache policy`, but we did cover that in our previous demo, so I'm going to jump ahead here. We still want to specify the exact version of Kubernetes we want. And for this demo, I'm using 1.29.1, and so we'll load that



variable here on line 86, and then we'll install the required Kubernetes packages and the specific versions with the code `apt-get install -y kubelet, kubeadm, and kubectl` specifying a version for each one of those packages. With the packages installed, let's go ahead and move down in the demo a little bit. And of course, if you want to install the latest and greatest, you would just omit the version parameter, and the code for that is on line 93. Before we move forward, let's make sure that kubelet and containerd are in the appropriate states before we move forward with joining this node to the cluster. And we'll check the `systemctl` status of `kubelet.service`, and it's in the inactive and dead state, which is good. And it's also enabled, which means when the system restarts that the kubelet will start automatically. And then similarly, I want to check containerd before we move on to make sure that it's active and running, and in the output at the bottom, we can see that it is, and also that it's enabled, and it is as well. It's going to break out of this output at the bottom and move forward. So we're logged into `c1-node1`. We installed some packages. We also configured some operating system parameters. Let's log out of `c1-node1` and back into `c1-cp1`. Back on `c1-cp1`, I can use the command `kubeadm token create --print-join-command` to generate the join command to join our node to this cluster. And so it's going to take this code at the bottom here, and put that into our clipboard because we're going to need that to paste into our worker node. So let's go ahead and SSH back into our worker node now because we want to execute that command on the worker node to join it to the cluster. On line 118 here, I'm going to take the command that we just generated and overwrite this command here. I'm going to format that so you can read it a little more easily. Put a new line there, and we'll also put a new line there. And so walking through this code from top to bottom, we have `sudo kubeadm join`, the IP address of our API server, `172.16.94.10`, and the port `6443`. On the next line, we have the bootstrap token. And on line 120, we have our `ca-cert-hash` so that we trust the certificate presented by the API server. So we'll take this code, we run this code on `c1-node1`, and it's going to join `c1-node1` to our cluster. Now, in the output at the bottom, we see it go through the various phases of the join process. We see the pre-flight checks, and we also see in the output that the certificate sign-in request was sent to the API server, and a response was received. So it's exchanged that information, and then it's configured the local kubelet on the system with the appropriate information to join the cluster. So now let's go ahead and log out of `c1-node1` back into `c1-cp1`, and we'll check the status of our nodes, and we can do that with `kubectl get nodes`. So what we see here is that `c1-cp1`, which is our control plane node, has a status of ready, and then we see our worker node, `c1-node1`, that has a status of not ready. The reason the node is currently in the not ready status is because the pod network hasn't finished deploying to this node yet. What's happening behind the scenes is that the containers are being pulled from their container registries and the pods that support the pod network are still starting up. Once that completes, the node's status will change to Ready. You can use the `--watch` parameter to pull the status of `kubectl get pods` to see the status of the pod network pods' deployments, and it'll update as things change. Running that here,



you can see that they're now all up and running and ready. And so if we check the status of our nodes with `kubectl get nodes`, you can see here that it's transitioned to Ready. And so that's how you join your first node to your cluster. Go ahead and perform the same exact steps into this demo on `c1-node2` and `c1-node3` in your cluster to get everything up and running. This cluster will be the foundation for the remaining courses in the series, so go ahead and get it all sorted before moving on to the next course, please.

Module Summary and What's Next!

All right, so here we are at the end of the course, and we covered a lot. We looked at some installation considerations to learn what you need to know before you install Kubernetes. Then we looked at the installation overview and we dove deep into the bootstrapping process for a Kubernetes cluster. And then we did several demos where we learned how to bootstrap a cluster and join nodes to a cluster using `kubeadm`. All right, so here we are at the end of the course, and I hope your Kubernetes learning journey is going well. In this course, we dove deep into how to install Kubernetes using `kubeadm`, and we finished up using `kubeadm` to install a basic cluster with a control plane node and three worker nodes. Before moving on to the next courses, you should complete this course and have your cluster up and running with that one control plane node and the three worker nodes. Well, I sure think we're off to a good start, and I hope you do, too. Please join me in the next course where we'll get started using your newly built cluster in the course *Working with Your cluster*. Thank you for watching!